SIMATS ENGINEERING

PYTHON PROGRAMMING – ASSIGNMENT 2

Control Flow, Strings and Functions

Student: Varshan J S

Register Number: 1924260130A

Institution: SIMATS Engineering

GitHub Repository: Add your repository URL here after creating it

Important: Re-run Q4 on your own computer, replace machine-dependent timing values with your measurements, and attach screenshots from your own terminal before submission.

Q1 – Decisions with Many Boundaries (6 Marks)

The program implements the electricity subsidy rule using independent if statements and an if–elif–else chain. It tests eight households, including the boundaries 100, 101, 200 and 201 units.

Predictions: 80 and 100 units receive full subsidy; 101 units with a ration card receive partial subsidy; 150 without a card receives none; 200 with a card receives partial subsidy; 201 without a senior member receives none; 250 with a member above 80 receives the special subsidy; 250 without that member receives none.

Separate if statements are independent and can produce more than one result if conditions overlap. An if–elif–else chain selects one matching branch. The chain is clearer for a single decision. With a fixed number of slabs, time and auxiliary space are O(1). A lookup list searched in a loop takes O(k) time for k slabs and O(k) storage; a match-case alternative would use guarded cases for ranges.

Q2 – Loops that Count Attempts (4 Marks)

The OTP program includes both while and for loop designs. Blank entries use continue and do not count. A correct OTP uses break. The loop else clause reports when the loop finishes without break. pass is a no-operation placeholder, demonstrated in a disabled branch.

A while loop is natural for interactive input that continues until success or lockout. A for loop is natural for a fixed sequence of attempts. Incrementing the counter before checking for a blank entry would incorrectly count blank entries. A correct security-question answer increases the attempt limit from three to five.

For a attempts, the loop takes O(a) time and O(1) auxiliary space. A recursive design would take O(a) time but O(a) stack space, so a loop is preferable.

Q3 – Vehicle Registration Number Validator (6 Marks)

The program normalises typed registrations by removing spaces and hyphens and converting letters to uppercase. It validates two letters, two digits, one or two series letters, and four final digits using slicing, isalpha() and isdigit(), without regular expressions. Valid values are formatted for a receipt, for example TN 09 AB 1234.

Python strings are immutable: replace() and upper() return new strings. Calling text.upper() without assigning or using the returned value does not modify text. For a 9-character registration, s[-4:] gives the final four digits and s[4:-4] gives one series letter. For a 10-character registration, s[-4:] gives the final four digits and s[4:-4] gives two series letters.

Both string-method validation and a regular-expression solution require O(L) time for a string of length L. The assignment's string-method approach is easy to trace and requires no regex.

Q4 – Building Long Text: + or join? (6 Marks)

Three implementations construct the same newline-separated statement: repeated +=, a list followed by join(), and generated f-string lines. The program measures 1,000, 10,000 and 200,000 lines and averages three runs.

Prediction: join() is generally preferred because it builds the final string in one operation. Repeated concatenation can cause repeated copying and approach $O(N^2)$ total work, while join() is O(N) in total output length. CPython can optimise some += patterns, so actual timings vary by version and workload. Parts and output require O(N) memory; streaming to a file or using StringIO may be appropriate for very large statements.

Measured timings are written to evidence/task04_string_building_output.txt. Replace these with measurements from your own computer before submission.

Q5 – Functions, Parameters and Scope (7 Marks)

The buggy add_item_buggy(item, order=[]) function reuses one list because the default object is created once when the function is defined. The corrected function uses order=None and creates a fresh list inside each call.

Positional arguments are matched by order; keyword arguments identify parameters by name; default arguments are used when omitted. calculate_bill() demonstrates a keyword-only GST rate and returns subtotal, GST and total as a tuple.

Local variables belong to a function. A global variable is defined at module level and requires the global keyword when reassigned inside a function. nonlocal modifies a variable in the nearest enclosing function scope. Global state is usually avoided because it creates hidden dependencies; parameters and return values are easier to test. Appending an item is O(1) amortised; an order with m items stores O(m) data.

Q6 – Recursion and Its Limits (8 Marks)

Let W(n) count the ways to climb n steps using 1-step or 2-step moves. Base cases: W(0)=1 and W(1)=1. Recursive case: W(n)=W(n–1)+W(n–2). Values for n=1 to 6 are 1, 2, 3, 5, 8 and 13.

Plain recursion repeats overlapping work and has exponential time complexity. Memoisation stores each computed state, reducing time to O(n), with O(n) cache space. It still has O(n) recursive call-stack depth. The program counts calls for n=25 and attempts n=1,000 to demonstrate that recursion may exceed Python's recursion limit.

Raising the recursion limit is not a robust deployment fix. An iterative method can compute the same recurrence in O(n) time and O(1) auxiliary space.

Q7 – Diagnose, Redesign and Validate (13 Marks)

Selected limitation: Q6's memoised recursive algorithm uses linear call-stack depth and may raise RecursionError for large n, even though it avoids repeated calculations.

Objective: compute the same result for n=1,000 and larger without recursion, using constant auxiliary memory.

Redesign: use bottom-up iteration and retain only the previous two results. Start a=1 and b=1. For each step from 2 through n, compute current=a+b, shift b=a and a=current, then return a.

Pseudocode: if n ≤ 1 return 1; set a=1, b=1; for step=2..n: current=a+b; b=a; a=current; return a.

Prediction: the iterative result will match memoised recursion for inputs within the recursive limit, while n=1,000 will complete without recursion failure. Both methods take O(n) time; memoisation uses O(n) cache and O(n) stack, while the iterative redesign uses O(1) auxiliary space.

Validation compares both methods for n=1, 2, 5, 10, 25, 100 and 1,000. The test output shows agreement where recursion completes and reports RecursionError for the recursive version at the largest input. The iterative method is therefore preferred for large deployments.

Overall Complexity Summary

Q1: O(1) time and O(1) space for fixed decision branches. Q2: O(a) time and O(1) extra space. Q3: O(L) time for registration length L. Q4: join is O(N); repeated += can approach $O(N^2)$. Q5: O(1) amortised per append and O(m) order storage. Q6: naive recursion exponential; memoised recursion O(n) time and O(n) space. Q7: iterative redesign O(n) time and O(1) auxiliary space.

Submission Checklist

Project includes one Python source file for each task, a README and captured text outputs. Run each program locally and capture terminal screenshots as additional evidence.

Create a GitHub repository named CSA08-1924260130A-Assignment2. Commit progressively, with at least one commit per task. Add the course faculty as collaborator if the repository is private. Put the working GitHub URL on the cover page and submit it in SIMATS.

Do not claim the included benchmark timings were measured on your own computer. Re-run Q4 locally and replace them with your own results.